PRODUCT REQUIREMENTS DOCUMENT (PRD)

1. Product overview

Product name: SwasthSetu (working title, rename freely)
Tagline: Check early. Reach care faster.
Platform: Android mobile application (React Native with Expo) with a FastAPI backend.
Languages: English, Hindi, Marathi.
Problem statement: AI-Powered Early Health Risk Detection and Healthcare Access Navigation.

SwasthSetu is a preventive triage and health navigation app. A person answers a short voice or icon-based intake, optionally records a smartphone heart rate reading, a cough sample and blood pressure readings, and receives:

1. An immediate emergency warning if red flags are present.
2. Explainable risk flags for hypertension, diabetes, cardiovascular disease and, for pregnant users, maternal risk.
3. A quality-checked camera heart rate and a cough screening flag.
4. A recommended triage level (Emergency, Urgent, Routine, Self-care).
5. The nearest suitable public facility, with distance, scheme eligibility, cost tier and wait-time tier.
6. A printable summary, a clinician SOAP note and a FHIR JSON export.

The app supports triage. It does not diagnose.

2. Problem background

In rural and semi-urban India, chronic illnesses such as hypertension, diabetes, cardiovascular disease and maternal complications are often found late. Reasons:

- Low medical literacy and little awareness of early warning signs.
- Shortage of doctors and specialists.
- People do not know which Primary Health Centre (PHC) or subsidized hospital to visit until a crisis happens.
- Language and literacy barriers in most digital health tools.

3. Goals and non-goals

Goals

- G1: Detect emergencies instantly and reliably.

- G2: Give early, explainable risk flags for hypertension, diabetes, cardiovascular disease and maternal risk.
- G3: Offer smartphone-based non-invasive screening: camera heart rate with a signal quality check, and cough analysis.
- G4: Make intake usable by low-literacy users through voice and icons in three languages.
- G5: Route the user to the nearest suitable facility with transparent distance, cost and scheme information.
- G6: Produce outputs that clinicians recognize (SOAP note) and systems can exchange (FHIR).
- G7: Be transparent about the AI: show model versions, metrics and limitations.

Non-goals

- Diagnosing disease or replacing a doctor.
- Prescribing medicines.
- Real integration with hospital booking systems or live wait-time feeds (simulated for the hackathon).
- Supporting every Indian language.

4. Target users

Persona 1: Sunita, 45, farmer's wife, rural Maharashtra. Speaks Marathi, limited reading ability, uses a low-cost Android phone, has occasional headaches and dizziness, has never checked her blood pressure.

Persona 2: Priya, 26, pregnant, second trimester. Lives in a semi-urban area, has swelling and headaches, does not know whether they are normal or dangerous.

Persona 3: Ramesh, ASHA worker. Visits households, owns a smartphone, wants a fast way to screen several people and refer them.

Persona 4: Dr. Meera, PHC medical officer. Sees many patients per day, wants a short structured note (SOAP) rather than a long story.

5. Scope

MVP (must build)

- Features F1 to F18 below.
- All six ML models: hypertension, CVD risk, diabetes, maternal risk, PPG signal quality classifier, cough classifier.

Stretch (build if time permits)

- Appointment booking polish, WhatsApp share, offline queue sync, on-device (ONNX or TFLite) versions of the signal quality and cough models.

Experimental (only if ahead of schedule)

- ASHA multi-patient mode, aggregate analytics dashboard.

6. Feature requirements

F1. Onboarding, language and consent

- User selects English, Hindi or Marathi on first launch. Language can be changed anytime.
- A short consent screen explains data use and states “This app gives screening guidance, not a diagnosis.”
- Guest mode is available. Login is optional but needed to keep history across devices.
- *Acceptance:* all screens render in the chosen language, and consent is stored with a timestamp.

F2. Conversational intake (voice and icons)

- One question per screen: age, sex, height, weight, smoking, alcohol, physical activity, family history, existing conditions, pregnancy status (if applicable).
- Symptom picker with large icon cards (chest pain, breathlessness, headache, dizziness, fever, cough, excessive thirst, frequent urination, swelling, blurred vision, bleeding in pregnancy and more) plus voice description.
- Every question is read aloud (text-to-speech) and can be answered by voice or tap.
- *Acceptance:* a user can finish intake in under 3 minutes without typing.

F3. Emergency red-flag detection

- A deterministic rule engine runs after each relevant answer and on final submit. It includes pregnancy-specific red flags.
- If triggered, a full-screen red alert shows “Go to the nearest hospital now” with a one-tap call to 108 and the nearest emergency-capable facility.
- Runs fully offline.
- *Acceptance:* all defined test cases trigger the alert in under 1 second.

F4. Blood pressure log

- Manual entry of systolic, diastolic and pulse with date and time.
- Trend chart and a BP category label.
- *Acceptance:* entries persist locally and sync when online.

F5. Camera heart rate (rPPG) with signal quality gate

- User holds the phone steady with the face in frame for 20 to 30 seconds. Live guidance shows lighting and movement tips.
- A **PPG signal quality classifier** scores the extracted signal in 10-second windows as Good, Marginal or Poor.
- Good: show BPM with a quality badge. Marginal: show BPM with a warning. Poor: do not show BPM, explain why (low light, movement, face lost) and offer retry or manual pulse entry.
- The result always carries a “screening estimate, not a medical device” note.
- *Acceptance:* mean absolute error under 5 BPM on windows classified Good, versus a pulse oximeter. Report error for Marginal and Poor windows too, to show the gate works.

F6. Cough screening

- User records a 10-second audio sample (one to three coughs) in a quiet place with the phone about 20 cm from the mouth.
- The app checks that a cough was detected and that background noise is acceptable. If not, it asks for a retry.
- The **cough classifier** returns a respiratory screening flag (Normal-like or Abnormal-like) with a confidence score. Optional: wet versus dry cough.
- The result is a supporting signal only. It can raise triage together with other symptoms (fever, breathlessness, cough lasting over 2 weeks) but never triggers an emergency on its own.
- Audio is processed and deleted. Only features and the result are stored.
- *Acceptance:* report AUC, sensitivity and specificity on a held-out set, and state clearly that the model is trained on public crowd-sourced data.

F7. Explainable risk prediction (chronic conditions)

- Three models: hypertension, diabetes and cardiovascular risk.
- Output per condition: probability, level (Low, Medium, High) and the top 3 contributing factors in plain language in the selected language.
- Each result shows the model version and a “How was this calculated?” link to the model card.
- *Acceptance:* every result includes at least 3 factors and the model version.

F8. Maternal risk module

- Shown only if the user is female and reports being pregnant.
- Extra questions: weeks of pregnancy, number of previous pregnancies, previous complications, swelling, headache and vision changes, reduced baby movement, body temperature (optional), blood sugar (optional).
- The maternal risk model uses age, systolic BP, diastolic BP, heart rate, and optionally blood sugar and body temperature. It returns Low, Mid or High risk with top factors.

- Pregnancy red flags (bleeding, severe headache with blurred vision, convulsions, reduced baby movement, BP at or above 160/110) are rule-based and override the model.
- High maternal risk leads to an Urgent triage and a facility search filtered to places with maternity services.
- *Acceptance:* all pregnancy red-flag test cases pass. Model metrics are reported per risk class.

F9. Triage result

- Combines red flags, risk levels, BP readings, heart rate, cough flag and symptoms into one triage level: Emergency, Urgent (see a doctor within 24 hours), Routine (visit a PHC within a week), Self-care (lifestyle guidance and recheck).
- Simple language, a colour code and an icon.

F10. Facility recommender

- Map with the user's location and ranked nearby facilities.
- Each facility card shows: name, type (PHC, CHC, district hospital, private), distance, Ayushman Bharat empanelment, cost tier (Free, Subsidized, Paid), wait-time tier (Short, Medium, Long, labeled simulated) and a call button.
- Ranking prefers emergency-capable facilities for Emergency and Urgent levels, maternity-capable facilities for high maternal risk, and the nearest PHC for Routine.
- *Acceptance:* results appear in under 3 seconds with network, and from cached data without network.

F11. Scheme eligibility check

- A short rule-based questionnaire estimates Ayushman Bharat PM-JAY eligibility and shows what to carry (Aadhaar, ration card and so on).
- Clearly labeled "indicative, verify at the facility."

F12. Appointment booking (simulated)

- User selects a facility and a slot and gets a confirmation screen with a reference ID.
- Marked "simulated for demo" on screen.

F13. Printable patient summary

- One-page PDF: patient details, symptoms, vitals, risk flags with reasons, triage level, recommended facility and disclaimer.
- Shared or printed from the phone.

F14. Clinician SOAP summary

- Subjective (symptoms, history), Objective (vitals, heart rate with quality, cough flag, maternal vitals if applicable), Assessment (risk flags, triage level), Plan (suggested next steps, referral).
- Viewable in-app and exportable as PDF.

F15. FHIR export

- Exports Patient, Observation (BP, heart rate, cough screening) and RiskAssessment resources as a FHIR R4 JSON Bundle.

F16. History

- Past assessments, BP trend and heart rate readings tied to the user account.

F17. Offline mode

- Intake, red-flag rules, BP log and cached facility list work without internet. Risk prediction, rPPG and cough analysis run when online (or are queued and run on reconnect).

F18. Model transparency

- A screen listing the six models with version, data source, headline metrics and limitations (model cards).

7. Non-functional requirements

- **Performance:** screen transitions under 300 ms, risk prediction under 2 seconds, rPPG and cough processing under 15 seconds after recording.
- **Device support:** Android 8 and above, 3 GB RAM phones.
- **Accessibility:** large touch targets, high contrast, icon plus text, voice prompts.
- **Privacy:** minimal data collection, encrypted transport, row-level security, no data sale, deletion on request. Videos and audio are deleted right after processing.
- **Reliability:** red-flag logic has no network dependency.
- **Localization:** all strings externalized in language files.

8. Success metrics

Model	What we report	Target (report the actual number)

Hypertension	AUC, recall for High, calibration	AUC above 0.75
CVD risk	AUC, recall for High, calibration	AUC above 0.75
Diabetes	AUC, recall for High, calibration	AUC above 0.75
Maternal risk	Accuracy, per-class recall (High-risk recall is the priority), confusion matrix	High-risk recall above 0.85
PPG signal quality	Accuracy, F1, drop in BPM error when Poor windows are rejected	F1 above 0.85
Cough classifier	AUC, sensitivity, specificity	AUC above 0.70 (public data is noisy)

Other metrics:

- Red-flag rules: 100% pass on the written test case list.
- rPPG: mean absolute error in BPM versus a pulse oximeter (10 or more people).
- Usability: 5 or more test users complete intake in under 3 minutes without help.
- Demo: full flow completed in under 3 minutes.

9. Assumptions and limitations

- Public datasets are used for training and may not represent rural Indian populations. This is stated openly.
- Chronic risk models use only features a rural user can provide (no lab values).
- The maternal dataset is small (about 1,000 records). The model is paired with deterministic clinical rules and is never the only safety layer.
- The cough model is trained on public crowd-sourced recordings made in varied conditions. It indicates “possible respiratory abnormality” and does not identify any disease.

- Diabetes and CVD datasets come from mixed or non-Indian populations. We report this, calibrate probabilities and cross-check with Indian risk tools where possible.
- rPPG accuracy depends on lighting, skin tone and movement. The quality classifier reduces but does not remove this risk.
- Facility wait times, booking and some scheme data are simulated.
- The app is a decision-support tool and is not a certified medical device.

10. Risks and mitigations

- *Scope too large:* build in the priority order in Document 5 and cut stretch items first.
- *rPPG unreliable:* quality gate, retry flow, manual pulse entry as a fallback.
- *Signal quality model lacks labelled data:* auto-label windows by comparing BPM to the pulse oximeter reference, and record our own 40 to 60 clips.
- *Maternal model overfits on small data:* cross-validation, simple models, calibrated thresholds and rule-based overrides.
- *Cough model weak in noisy rooms:* audio quality gate, retry flow, and a “supporting signal only” role.
- *Small or biased datasets:* calibration, honest limitations, favour recall for high-risk flags.
- *Marathi voice recognition inconsistent:* icon input is always available as the fallback.
- *Medical liability perception:* consistent “screening guidance, not diagnosis” messaging.

11. Demo scenarios

1. **Emergency:** a user selects chest pain and sweating, and the red alert plus the nearest hospital appears.
2. **Chronic risk (Sunita):** age 45, high BMI, family history of diabetes, headaches. Medium hypertension risk and High diabetes risk with explanations, a quality-checked heart rate, a cough flag, a PHC recommendation and a PDF summary.
3. **Maternal (Priya):** pregnant, raised BP and swelling. Maternal risk High, Urgent triage, nearest maternity-capable facility.
4. **Clinician view:** the same case as a SOAP note and FHIR JSON.
1.

TECHNICAL REQUIREMENTS DOCUMENT (TRD)

1. System architecture

Principle: anything safety-critical (red flags) runs on the device. Anything compute-heavy (ML, rPPG, audio) runs on the backend.

2. Tech stack

Mobile

- React Native with Expo (TypeScript), development builds (needed for native voice, camera and audio modules).
- Navigation: Expo Router or React Navigation.
- State: Zustand. Server state: TanStack Query.
- Styling: NativeWind (Tailwind) or StyleSheet with a shared theme.
- Localization: i18next with `en.json`, `hi.json`, `mr.json`.
- Voice input: `@react-native-voice/voice` (Android SpeechRecognizer, locales en-IN, hi-IN, mr-IN).
- Voice output: `expo-speech`. Fallback: pre-recorded audio clips for key prompts if a Marathi voice is not installed.
- Camera: `expo-camera` (video recording).
- Audio recording: `expo-av` or `expo-audio` (cough recorder).
- Maps: Leaflet inside `react-native-webview` with OpenStreetMap tiles (no API key), or `react-native-maps`.

- Location: `expo-location`.
- Local storage: `expo-sqlite` and `expo-secure-store`.
- PDF and sharing: `expo-print`, `expo-sharing`.
- Charts: `react-native-svg` with a small chart library.

Backend

- Python 3.11, FastAPI, Uvicorn, Pydantic.
- ML: scikit-learn, XGBoost, SHAP, pandas, joblib, imbalanced-learn (optional).
- rPPG: OpenCV, MediaPipe, NumPy, SciPy.
- Audio: librosa, soundfile, pydub (needs ffmpeg).
- Optional: onnxruntime for exported models.
- DB client: `supabase-py` or SQLAlchemy with asyncpg.
- Hosting: Google Cloud Run, Render or Railway (free tier is fine for the demo).

Data and platform

- Supabase PostgreSQL with the PostGIS extension, Supabase Auth, Supabase Storage (private bucket for temporary uploads, auto-deleted after processing).

Tools

- Git and GitHub, Expo EAS Build (APK), Postman or Thunder Client, Jupyter or Colab for model training, Figma for UI, pytest, Jest.

3. Model inventory

#	Model	Task	Input	Primary algorithm	Output	Dataset candidates (verify licence and fields)
1	Hypertension	Binary risk, 3 bands	Age, sex, BMI, lifestyle, family history, symptoms, optional BP	XGBoost (baseline Logistic Regression)	Probability, band, SHAP	NHANES extract, Kaggle hypertension or CVD datasets

2	CVD risk	Binary risk, 3 bands	Age, sex, BMI, BP, smoking, activity, diabetes status, symptoms	XGBoost or Random Forest	Probability, band, SHAP	Kaggle Cardiovascular Disease (70k), UCI Heart Disease, Framingham
3	Diabetes	Binary risk, 3 bands	Age, sex, BMI, activity, family history, BP, symptoms	XGBoost or Random Forest	Probability, band, SHAP	CDC BRFSS Diabetes Health Indicators, Pima Indians (needs glucose), NHANES
4	Maternal risk	3-class (Low, Mid, High)	Age, systolic, diastolic, heart rate, optional blood sugar, optional body temp	Random Forest or XGBoost, calibrated	Class probabilities , SHAP	UCI Maternal Health Risk
5	PPG signal quality	3-class (Good, Marginal, Poor)	Feature vector from a 10 s rPPG window	Random Forest or XGBoost, optional 1D CNN	Class and score	UBFC-rPPG, PURE, MMPD, plus our own labelled clips
6	Cough classifier	Binary (Normal-like or Abnormal-like), optional wet or dry	10 s audio, MFCC and spectral features	XGBoost on MFCC statistics, optional small CNN on log-mel	Probability and class	COUGHVID, Coswara, ICBHI respiratory sounds

Rules for all models

- Train on features a rural user can provide without labs.
- Split by patient or recording, never by random row, to avoid leakage.
- Calibrate probabilities. Report AUC, precision, recall, F1, confusion matrix and calibration curve.

- Save `model.joblib`, `metrics.json`, `model_card.md` and a `version` string per model.
- SHAP for models 1 to 4. Feature importance and example plots for models 5 and 6.

4. Emergency red-flag rule engine (on device)

A pure TypeScript function takes the intake object and returns a list of triggered rules.

Initial rule set (review against standard triage guidance):

- RF01: Chest pain or pressure with sweating, nausea, arm or jaw pain, or breathlessness.
- RF02: Systolic BP at or above 180 or diastolic at or above 120.
- RF03: Sudden weakness on one side, face drooping, slurred speech or sudden vision loss (stroke signs).
- RF04: Severe breathlessness at rest.
- RF05: Pregnancy with any of: vaginal bleeding, severe headache with blurred vision, convulsions, reduced baby movement, or BP at or above 160/110.
- RF06: Fainting or loss of consciousness.
- RF07: Diabetic patient with confusion, extreme drowsiness, fruity breath or persistent vomiting.
- RF08: Measured heart rate above 150 or below 40 at rest with symptoms (only if signal quality is Good).
- RF09: Coughing blood.
- RF10: Seizure.

Each rule has an ID, condition, translated message key and severity. Unit tests cover every rule and boundary values.

5. Risk prediction service (models 1 to 4)

Shared pipeline

1. Clean each dataset and map it to a shared feature schema.
2. Train Logistic Regression (baseline), Random Forest and XGBoost. Pick by cross-validated AUC and recall for the High band.
3. Handle imbalance with class weights.
4. Calibrate probabilities (`CalibratedClassifierCV`).
5. Risk bands: Low below 0.33, Medium 0.33 to 0.66, High above 0.66. Tune thresholds to favour recall for High.
6. Compute SHAP values per prediction and return the top 3 factors.
7. Save model, metrics and model card with a version string.

Model-specific notes

- **Hypertension:** if the dataset label is derived from blood pressure, do not use BP as an input (label leakage). Train a “no BP” model for users without a reading and a “with BP” model for users who log BP. Repeated readings at or above 140/90 are flagged by rules, not ML.
- **CVD risk:** train on the 70k Kaggle dataset, use UCI Heart Disease as a small external check, and compare against a simple WHO/ISH-style chart or Framingham score as a baseline. Report both.
- **Diabetes:** train the main model on lifestyle-based BRFSS indicators because it needs no lab values. Use Pima only for an optional “with glucose” clinician-view model. Cross-check with the Indian Diabetes Risk Score (age, waist, activity, family history) as a rule-based baseline.
- **Maternal risk:** the UCI dataset is small. Use stratified cross-validation, keep the model simple, optimise for High-risk recall, and handle missing blood sugar and temperature (XGBoost handles missing values, or train a reduced-feature variant). Layer deterministic pregnancy rules on top so the model is never the only safety net.

SHAP to plain language: a mapping file turns `bmi_high` into “Weight is higher for your height” and so on, in three languages.

6. rPPG service

Input: a 20 to 30 second face video (portrait, 30 fps, 480p to 720p) uploaded as multipart.

Pipeline

1. Extract frames with OpenCV.
2. Detect the face and landmarks with MediaPipe. Define regions of interest (forehead and both cheeks).
3. For every frame compute the mean R, G, B in the ROI, plus brightness and landmark movement.
4. Build the signal with the green channel or the POS method.
5. Detrend, normalize, and bandpass filter (0.7 to 4.0 Hz).
6. Split into 10-second windows with 5-second stride.
7. For each window compute quality features and run the **PPG Signal Quality Classifier**.
8. For Good and Marginal windows, estimate BPM with Welch’s method or FFT. Combine across windows using the median.
9. If all windows are Poor, return `quality = poor` with reasons and no BPM.
10. Delete the video. Store only BPM, quality and metadata.

Validation protocol: record rPPG and a pulse oximeter reading at the same time for 10 or more people in different lighting. Report mean absolute error, Bland-Altman limits and failure rate, with and without the quality gate.

7. PPG Signal Quality Classifier (model 5)

Purpose: stop wrong heart rate readings from reaching the user by rejecting unreliable signals.

Features per 10 s window

- Spectral: SNR at the dominant peak, peak-to-total power ratio, spectral entropy, harmonic ratio.
- Time domain: skewness, kurtosis, zero-crossing rate, autocorrelation peak height and lag, number of detected peaks and regularity of peak intervals.
- Context: mean ROI brightness, brightness variance, face-detected fraction, landmark motion magnitude.

Labels

- Public datasets (UBFC-rPPG, PURE, MMPD) include reference pulse. Label a window Good if estimated BPM is within 3 BPM of the reference, Marginal if within 3 to 8 BPM, Poor if worse than 8 BPM or the face is lost.
- Add our own clips: 40 to 60 recordings from different people and lighting with a simultaneous pulse oximeter reading.

Model: Random Forest or XGBoost on the features. Optional second version: small 1D CNN on the raw window.

Evaluation: accuracy, macro F1, confusion matrix, and the key slide result: “BPM error with the gate versus without it.”

Runtime: inference takes milliseconds on the backend. Stretch goal: export to ONNX and run on the phone for instant “move or relight” feedback while recording.

8. Cough service and classifier (model 6)

Input: a 10-second mono audio clip at 16 kHz or higher (WAV or M4A), uploaded as multipart.

Pipeline

1. Convert to 16 kHz mono and normalize.
2. Quality check: estimate signal-to-noise ratio and clipping. Reject silence or clipping.
3. Cough detection: energy-based segmentation, optionally a small cough detector. If no cough event is found, return `no_cough_detected` and ask for a retry.
4. Extract features for each cough segment: 20 to 40 MFCCs with deltas, spectral centroid, bandwidth, rolloff, zero-crossing rate, RMS energy, duration. Aggregate with mean and standard deviation.
5. Classify with XGBoost (baseline) and optionally a small CNN on log-mel spectrograms.
6. Average probabilities across detected segments.
7. Delete the audio. Store the result and features only.

Classes: Normal-like versus Abnormal-like (symptomatic). Optional second head: wet versus dry, if the dataset labels allow it.

Training data: COUGHVID (crowd-sourced with expert labels) as the main source, Coswara or ICBHI as extra data. Deduplicate by speaker, filter low-quality recordings using the dataset's quality labels, and augment with noise, pitch shift and time stretch.

Output example

json

```
{"cough_detected": true, "segments": 2, "flag": "abnormal_like", "probability": 0.64, "quality": "ok", "model_version": "2026.10.1"}
```

Use in triage: supporting signal only. A cough flag together with fever, breathlessness or cough over 2 weeks raises the triage level. It never causes an emergency alert by itself.

9. Facility recommender

Data: curated `facilities` table for the demo region (Nagpur and nearby districts), seeded from OpenStreetMap (Overpass API for hospitals and clinics), government facility directories and the PM-JAY hospital list. Fields such as cost tier and wait-time tier are derived or simulated and flagged `is_simulated = true`.

Ranking score (example)

```
score = w1*distance_norm + w2*type_match + w3*scheme_match + w4*wait_tier + w5*open_now
```

- Emergency: only emergency-capable facilities, sorted by distance.
- Urgent: CHC, district hospital or empanelled private hospital, prefer near and open.
- Maternal High risk: only facilities where `services` includes `maternity` (and emergency-capable if red flags exist).
- Routine: nearest PHC, prefer free.
- Query uses PostGIS `ST_DWithin` and `ST_Distance`. Fallback: Haversine in Python.

10. Scheme eligibility service

Rules are stored as JSON and evaluated in Python. Sample rules cover Ayushman Bharat PM-JAY, including the extension for senior citizens aged 70 and above, plus a household-based deprivation questionnaire. Output: Likely eligible, Possibly eligible, or Not likely, plus documents to carry. Always labeled indicative. Verify current criteria on the official PM-JAY site before the final demo.

11. Report services

SOAP generator: template-based and deterministic.

- S: chief complaints, duration, history, pregnancy details.
- O: vitals (BP, heart rate with quality label, cough flag, BMI, maternal vitals).
- A: red flags, risk flags with probabilities, triage level.
- P: recommended action, referral facility, follow-up interval, lifestyle advice.

FHIR R4 mapping

- `Patient`: gender, birthDate (or age extension), language.
- `Observation` (BP): LOINC 85354-9 with components for systolic (8480-6) and diastolic (8462-4).
- `Observation` (heart rate): LOINC 8867-4, with a note carrying the signal quality label.
- `Observation` (cough screening): project-defined code system, `valueString` (`normal_like` or `abnormal_like`), `note` with probability and disclaimer.
- `RiskAssessment`: one per condition including maternal, with `qualitativeRisk` and `prediction.probabilityDecimal`.
- Wrapped in a `Bundle` of type `collection`.
- Validate output against the public HAPI FHIR validator before the demo.

12. API specification (v1)

Base path: `/api/v1`. All endpoints except `/health` need `Authorization: Bearer <Supabase JWT>`. Guest users use Supabase anonymous sign-in, so they also get a JWT.

Method	Endpoint	Purpose
GET	/health	Service health check
POST	/assessments	Create an assessment with intake data
GET	/assessments	List the user's assessments
GET	/assessments/{id}	Full assessment with results
POST	/assessments/{id}/redflags	Server-side red-flag check (mirror of on-device rules)
POST	/risk/predict	Accepts `conditions: ["hypertension","diabetes","cvd","maternal"]`, returns all requested results
POST	/maternal/assess	Saves pregnancy details, runs the maternal model plus pregnancy rules

POST	/vitals/bp	Save a BP reading
GET	/vitals/bp	BP history
POST	/vitals/rppg	Upload video. Returns BPM (if quality allows), quality label, quality score, reasons
POST	/vitals/cough	Upload audio. Returns cough detected, flag, probability, quality
GET	/facilities/nearby	Query: lat, lng, radius_km, type, scheme, triage_level, service
POST	/schemes/eligibility	Returns eligibility result
POST	/appointments	Create a simulated appointment
GET	/assessments/{id}/soap	SOAP note JSON
GET	/assessments/{id}/fhir	FHIR Bundle JSON
GET	/assessments/{id}/summary	Patient summary JSON (PDF is rendered on device)
GET	/models	Lists the six models with version, training date, dataset, headline metrics
GET	/models/{name}/card	Returns the model card
DELETE	/me	Delete the account and data

Example: POST /risk/predict request

json

```
{
  "assessment_id": "uuid",
  "conditions": ["hypertension", "diabetes", "cvd"],
  "features": {
    "age": 45, "sex": "F", "height_cm": 155, "weight_kg": 72,
    "smoker": false, "alcohol": false, "activity_level": "low",
    "family_history_diabetes": true, "family_history_cvd": false,
    "systolic": 142, "diastolic": 92,
    "symptoms": ["headache", "dizziness", "frequent_urination"]
  },
  "language": "mr"
}
```

Response

json


```
{
  "results": [
    {
      "condition": "hypertension",
      "probability": 0.58,
      "level": "medium",
      "model_version": "2026.10.1",
      "top_factors": [
        {"feature": "systolic", "impact": 0.21, "text": "Your upper BP reading is higher than normal"},
        {"feature": "bmi", "impact": 0.14, "text": "Weight is higher for your height"},
        {"feature": "age", "impact": 0.09, "text": "Risk rises with age"}
      ]
    }
  ],
  "triage_level": "routine"
}
```


Example: maternal result

json


```
{
  "condition": "maternal",
  "level": "high",
  "class_probabilities": {"low": 0.08, "mid": 0.22, "high": 0.70},
  "top_factors": [
    {"feature": "systolic", "text": "Upper BP reading is high for pregnancy"},
    {"feature": "blood_sugar", "text": "Blood sugar is higher than expected"}
  ],
  "rule_overrides": []
}
```


Example: POST /vitals/rppg response

json


```
{
  "bpm": 78,
  "quality": "good",
  "quality_score": 0.91,
  "windows": [
    {"start_s": 0, "label": "good"},
    {"start_s": 5, "label": "good"},
    {"start_s": 10, "label": "marginal"}
```

```
  ],
  "reasons": [],
  "model_version": "2026.10.1"
}
```

13. Security and privacy

- HTTPS only. JWT verified on the backend using the Supabase JWT secret.
- Row Level Security on every user-owned table.
- Secrets live in environment variables, never in the app bundle (the Supabase anon key is the only public key).
- Videos and audio are deleted after processing, inside a `finally` block.
- Consent record is stored. A delete-my-data endpoint exists.
- No third-party analytics SDKs.
- Medical disclaimer on result screens and PDFs.

14. Offline strategy

- Bundle the rule engine, translations, symptom catalog and the facility list for the demo region in the app.
- Store assessments and BP entries in SQLite with a `synced` flag. Sync when online.
- Risk, rPPG and cough calls are queued when offline, and a message explains that results will appear when the phone reconnects.

15. ML deployment notes

- Load all six models once at startup from `MODEL_DIR` and expose their versions through `/models`.
- Limit upload size (for example 15 MB for video and audio) and stream to temporary storage.
- Backend dependencies for ML: `xgboost scikit-learn shap pandas joblib librosa soundfile pydub scipy opencv-python mediapipe`. Optional: `imbalanced-learn onnxruntime`.

16. Environment configuration

```
SUPABASE_URL=
SUPABASE_ANON_KEY=
SUPABASE_SERVICE_ROLE_KEY=      (backend only)
SUPABASE_JWT_SECRET=            (backend only)
API_BASE_URL=
MODEL_DIR=./models
MAX_UPLOAD_MB=15
```

APP FLOW

1. Screen inventory

1. Splash
2. Language selection
3. Consent and disclaimer
4. Login or Continue as guest
5. Home
6. Intake: Basic info (age, sex, height, weight)
7. Intake: Lifestyle (smoking, alcohol, activity)
8. Intake: History (existing conditions, family history, pregnancy)
9. Intake: Pregnancy details (only if female and pregnant)
10. Intake: Symptoms (icon grid and voice)
11. Intake: Symptom details (duration, severity)
12. Emergency Alert (full-screen overlay)
13. Vitals hub
14. BP entry and BP log
15. Heart rate scan with live quality feedback (lighting, movement, face lost)
16. Heart rate result with quality badge, or “Poor signal, retry” screen
17. Cough recording (instructions, 10 s recorder, level meter)
18. Cough result (flag, confidence, disclaimer) or retry screen
19. Processing screen
20. Result: triage level and risk cards
21. Risk detail (SHAP factors)
22. Maternal risk result and detail
23. Facility list and map
24. Facility detail
25. Scheme eligibility questionnaire and result
26. Appointment booking (simulated) and confirmation
27. Summary (patient PDF preview)
28. Clinician view (SOAP note)
29. FHIR export
30. History
31. Settings (language, delete data, about)
32. Model transparency and model cards

2. User journeys

Journey A: Standard assessment (Sunita)

1. Open the app, choose Marathi, accept consent, continue as guest.
2. Home: tap “Start health check.”

3. Answer basic info, lifestyle and history screens (voice or tap).
4. Select symptoms (headache, dizziness) and set the duration.
5. The app runs the red-flag check. None found, so continue.
6. Vitals hub: add a BP reading, record a 25-second heart rate scan (the quality classifier approves the signal), and record a 10-second cough sample.
7. Processing screen, then the Result screen: triage level “Routine, visit a PHC within a week”, with risk cards for hypertension (Medium), diabetes (High) and cardiovascular (Low), plus the cough screening flag.
8. Tap a risk card to see the top factors in simple language.
9. Tap “Find care”: map and list of nearest PHCs and Ayushman Bharat hospitals.
10. Check scheme eligibility and book a simulated appointment.
11. Download the PDF summary and share it. History is saved.

Journey B: Emergency path

1. During the symptoms screen the user selects chest pain and sweating.
2. Rule RF01 fires and the full-screen red Emergency Alert appears immediately.
3. Actions: Call 108, Navigate to nearest emergency facility, Continue assessment anyway.
4. The assessment is saved with `triage_level = emergency`.

Journey C: Clinician view

1. From the Result screen, tap “Clinician summary.”
2. View the SOAP note, then export as PDF or FHIR JSON.

Journey D: Pregnant user (Priya)

1. The user selects female and pregnant on the history screen.
2. The Pregnancy details screen appears: weeks, previous pregnancies, previous complications, swelling, headache, vision changes, baby movement.
3. Pregnancy red flags run immediately on the device. If triggered, the Emergency Alert opens.
4. Vitals hub asks for BP, heart rate (camera), and optionally temperature and blood sugar.
5. Result shows the Maternal risk card (Low, Mid or High) with reasons, plus the other risk cards.
6. High maternal risk gives Urgent triage, with “Find maternity care” showing facilities that offer maternity services.

3. Navigation map

- Splash
- └─ Language Select
- └─ Consent
- └─ Auth (Login | Guest)
- └─ Home (Bottom tabs)
- ├─ Tab 1: Check (Start assessment)
- │ └─ Intake Stack
- │ Basic → Lifestyle → History
- │ → [if pregnant] Pregnancy Details
- │ → Symptoms → Symptom Details
- │ ├─ (red flag) → Emergency Alert → [Call 108 | Find hospital | Continue]
- │ └─ Vitals Hub
- │ ├─ BP Entry
- │ ├─ Heart Rate Scan → Quality check → (Good/Marginal) Result | (Poor) Retry
- │ └─ Cough Recording → Quality check → Cough Result | Retry
- │ └─ Processing → Result
- │ ├─ Risk Detail
- │ ├─ Maternal Risk Detail
- │ ├─ Find Care → Facility List/Map → Facility Detail
- │ │ ├─ Scheme Eligibility
- │ │ └─ Book Appointment (simulated) → Confirmation
- │ ├─ Summary (PDF)
- │ └─ Clinician View (SOAP) → FHIR Export
- ├─ Tab 2: Vitals (BP log, heart rate history)
- ├─ Tab 3: Care (Facility map, direct access)
- ├─ Tab 4: History (Past assessments)

└─ Tab 5: Settings (Language, Data deletion, About, Model Transparency → Model Card)

4. Triage decision logic

Evaluated in this order (first match wins):

1. Any red flag (including pregnancy red flags) triggered gives **Emergency**.
2. Maternal risk High, or any chronic condition at High risk with a relevant symptom, or BP at or above 160/100, gives **Urgent**.
3. Any condition at Medium or High risk, or an abnormal-like cough combined with fever, breathlessness or cough lasting over 2 weeks, gives **Routine** (or **Urgent** if breathlessness is moderate or severe).
4. Otherwise **Self-care** with lifestyle guidance and a recheck reminder.

The cough flag alone never moves the user above Self-care. It adds a note and a suggestion to recheck. Thresholds are configurable in a JSON file and should be reviewed with a medical advisor if one is available.

5. Edge cases and states

- **No internet:** red flags, intake and BP log work. Risk, rPPG and cough show “Will complete when online.”
- **Location denied:** ask for the district manually from a list.
- **Voice not recognized:** show “Try again or tap an icon.”
- **rPPG poor quality:** show the specific reason from the classifier (too dark, too much movement, face not visible), a retry button, and manual pulse entry.
- **No cough detected:** “We could not hear a cough. Please cough 2 to 3 times close to the phone.”
- **Noisy room:** cough quality fails, suggest moving to a quiet place.
- **Microphone or camera permission denied:** explain why it is needed and allow skipping that step.
- **Pregnant user without blood sugar or temperature:** the maternal model runs with the available inputs, with a note that results may improve with those readings.
- **Marathi voice not installed:** use recorded audio clips or fall back to on-screen text.
- **API failure:** show cached results where possible and a retry option.
- **Unusual inputs:** validate ranges (for example age 1 to 120, systolic 60 to 260).

6. Screen design rules

- One question per screen, large buttons, minimum touch target of 48 dp.
- Always show icon plus text. Speaker button on every prompt.
- Colour code: green (self-care), amber (routine), orange (urgent), red (emergency). Do not rely on colour alone, so add icons and labels.
- Persistent “Call 108” access on the Emergency screen.
- Consistent disclaimer footer on results.
- Cough and heart rate screens show a clear “screening estimate” label.
-

BACKEND SCHEMA

1. Overview

Database: Supabase PostgreSQL with the `postgis` and `pgcrypto` extensions. All user-owned tables have Row Level Security.

2. Enums

- `user_role`: patient, asha, clinician
- `sex_type`: male, female, other
- `triage_level`: emergency, urgent, routine, self_care
- `risk_level`: low, medium, high
- `condition_type`: hypertension, diabetes, cardiovascular, maternal
- `facility_type`: phc, chc, sub_centre, district_hospital, medical_college, private
- `cost_tier`: free, subsidized, paid
- `wait_tier`: short, medium, long
- `appointment_status`: booked, cancelled, completed
- `signal_quality`: good, marginal, poor
- `cough_flag`: normal_like, abnormal_like, no_cough_detected, poor_audio

3. Tables

profiles (1 to 1 with `auth.users`)

- id uuid PK (references auth.users.id)
- full_name text
- phone text
- birth_year int
- sex sex_type
- height_cm numeric
- weight_kg numeric
- preferred_language text (en, hi, mr)
- district text
- state text
- role user_role default ‘patient’
- created_at timestamptz

patient_history (1 to 1 with profiles)

- id uuid PK

- profile_id uuid FK to profiles.id (unique)
- known_hypertension boolean
- known_diabetes boolean
- known_heart_disease boolean
- smoker boolean
- alcohol boolean
- activity_level text (low, moderate, high)
- family_history_diabetes boolean
- family_history_cvd boolean
- is_pregnant boolean
- pregnancy_weeks int
- updated_at timestamptz

consents

- id uuid PK
- profile_id uuid FK to profiles.id
- consent_type text
- granted boolean
- granted_at timestamptz

symptom_catalog (reference data, public read)

- code text PK
- label_en text
- label_hi text
- label_mr text
- icon text
- body_system text
- is_red_flag_candidate boolean

assessments

- id uuid PK
- profile_id uuid FK to profiles.id
- language text
- input_mode text (voice, icon, mixed)
- intake_snapshot jsonb (answers at the time of assessment)
- latitude numeric
- longitude numeric
- red_flag_triggered boolean default false
- triage_level triage_level
- status text (in_progress, completed)
- started_at timestamptz
- completed_at timestamptz

assessment_symptoms

- id uuid PK
- assessment_id uuid FK to assessments.id (on delete cascade)
- symptom_code text FK to symptom_catalog.code
- severity int (1 to 3)
- duration_days int
- free_text text

maternal_checks

- id uuid PK
- assessment_id uuid FK to assessments.id (unique, on delete cascade)
- profile_id uuid FK to profiles.id
- gestational_weeks int
- gravida int (number of pregnancies)
- previous_complications boolean
- swelling boolean
- severe_headache boolean
- blurred_vision boolean
- reduced_fetal_movement boolean
- vaginal_bleeding boolean
- body_temp_c numeric (nullable)
- blood_sugar_mgdl numeric (nullable)
- rule_overrides text[] (rule codes that fired)
- created_at timestamptz

vitals_bp

- id uuid PK
- profile_id uuid FK to profiles.id
- assessment_id uuid FK to assessments.id (nullable)
- systolic int
- diastolic int
- pulse int (nullable)
- measured_at timestamptz

vitals_rppg

- id uuid PK
- profile_id uuid FK to profiles.id
- assessment_id uuid FK to assessments.id
- bpm numeric (nullable, null when quality is poor)
- quality signal_quality
- quality_score numeric (0 to 1)

- window_results jsonb (label per window)
- quality_reasons text[]
- duration_s int
- method text (green, pos)
- reference_bpm numeric (nullable, for validation sessions)
- quality_model_version text
- measured_at timestamptz

vitals_cough

- id uuid PK
- profile_id uuid FK to profiles.id
- assessment_id uuid FK to assessments.id
- flag cough_flag
- probability numeric
- segments_detected int
- audio_quality text (ok, noisy, too_quiet, clipped)
- cough_type text (nullable: wet, dry)
- model_version text
- measured_at timestamptz

risk_results

- id uuid PK
- assessment_id uuid FK to assessments.id (on delete cascade)
- condition condition_type
- probability numeric
- level risk_level
- class_probabilities jsonb (used for maternal)
- top_factors jsonb
- model_version text
- created_at timestamptz

red_flag_events

- id uuid PK
- assessment_id uuid FK to assessments.id (on delete cascade)
- rule_code text
- description text
- triggered_at timestamptz

ml_models (model registry, public read)

- id uuid PK
- name text (hypertension, cvd, diabetes, maternal, ppg_quality, cough)
- version text

- algorithm text
- trained_on text (dataset names)
- metrics jsonb
- model_card_md text
- trained_at timestamptz
- is_active boolean

facilities (reference data, public read)

- id uuid PK
- name text
- type facility_type
- address text
- district text
- state text
- location geography(Point, 4326)
- phone text
- services text[] (for example maternity, emergency, ncd_clinic, lab)
- emergency_capable boolean
- is_pmjay_empanelled boolean
- cost_tier cost_tier
- wait_tier wait_tier
- opening_hours jsonb
- data_source text
- is_simulated boolean default true

schemes (reference data)

- id uuid PK
- code text unique (for example PMJAY)
- name text
- description text
- eligibility_rules jsonb

facility_schemes (many to many)

- facility_id uuid FK to facilities.id
- scheme_id uuid FK to schemes.id
- primary key (facility_id, scheme_id)

appointments (simulated)

- id uuid PK
- profile_id uuid FK to profiles.id
- facility_id uuid FK to facilities.id
- assessment_id uuid FK to assessments.id (nullable)

- slot_start timestamptz
- status appointment_status default ‘booked’
- reference_code text
- is_simulated boolean default true
- created_at timestamptz

4. Relationships

- auth.users 1 ── 1 profiles 1 ── 1 patient_history
- profiles 1 ── N assessments
- profiles 1 ── N consents
- profiles 1 ── N vitals_bp
- profiles 1 ── N appointments
- assessments 1 ── N assessment_symptoms N ── 1 symptom_catalog
- assessments 1 ── 0..1 maternal_checks
- assessments 1 ── N vitals_rppg
- assessments 1 ── N vitals_cough
- assessments 1 ── N risk_results
- assessments 1 ── N red_flag_events
- facilities N ── N schemes (via facility_schemes)
- facilities 1 ── N appointments

ml_models: referenced by version strings in risk_results, vitals_rppg and vitals_cough

5. Indexes

- `facilities` GIST index on `location`.
- `assessments (profile_id, started_at desc)`.
- `vitals_bp (profile_id, measured_at desc)`.
- `vitals_rppg (assessment_id)` and `vitals_cough (assessment_id)`.
- `risk_results (assessment_id)`.

6. Row Level Security

- Enable RLS on profiles, patient_history, consents, assessments, assessment_symptoms, maternal_checks, vitals_bp, vitals_rppg, vitals_cough, risk_results, red_flag_events, appointments.
- Policy pattern for direct-owner tables:

sql

- create policy "own rows" on assessments

for all using (auth.uid() = profile_id) with check (auth.uid() = profile_id);

- Child tables check ownership through the parent:

sql

```
create policy "own symptoms" on assessment_symptoms
for all using (exists (
  select 1 from assessments a
  where a.id = assessment_id and a.profile_id = auth.uid()
));
```

- `symptom_catalog`, `facilities`, `schemes`, `facility_schemes`, `ml_models`: read for everyone, no write from clients.
- The backend uses the service role key only for server operations and still scopes queries by the verified user ID.

7. Authentication flow

1. The app launches and checks for an existing Supabase session in secure storage.
2. If none exists, the user picks **Continue as guest** (Supabase anonymous sign-in) or signs in with email OTP (phone OTP is optional because it needs an SMS provider).
3. Supabase returns a JWT (access token) and a refresh token, stored in `expo-secure-store`.
4. The app sends `Authorization: Bearer <access_token>` to FastAPI.
5. FastAPI verifies the signature with the Supabase JWT secret, reads `sub` as the user ID and attaches it to the request.
6. Guest accounts can later be upgraded by linking an email, so history is kept.
7. Logout clears secure storage. Delete account calls `DELETE /me`, which removes the user's rows and the auth record.

8. Data flow (one assessment)

1. The app creates an `assessments` row (status in_progress) and stores the intake locally.
2. Red-flag rules run on the device. Any trigger writes `red_flag_events` and sets `red_flag_triggered`.
3. BP entries go to `vitals_bp`.
4. The rPPG video is uploaded to FastAPI, quality-classified and processed. Only the result goes to `vitals_rppg`, and the video is deleted.
5. The cough audio is uploaded, quality-checked and classified. Only the result goes to `vitals_cough`, and the audio is deleted.
6. If the user is pregnant, the pregnancy details go to `maternal_checks` and pregnancy rules run before the maternal model.

7. `POST /risk/predict` loads the models, computes probabilities and SHAP factors, and saves `risk_results`.
8. Triage logic sets `assessments.triage_level` and marks the assessment completed.
9. `GET /facilities/nearby` returns ranked facilities using PostGIS (with the maternity filter when needed).
10. SOAP, FHIR and summary endpoints read the stored rows and format the outputs.

9. Seed data

- 25 to 40 symptoms in `symptom_catalog` with three languages.
- 30 to 80 facilities around Nagpur and nearby districts (PHCs, CHCs, district and empanelled hospitals) with `services` filled in (maternity, emergency, ncd_clinic, lab).
- Schemes: PM-JAY and optionally state schemes.
- Six rows in `ml_models` with versions, metrics and model cards.
- A demo user and 3 sample assessments (chronic risk, emergency, maternal) for the presentation.

IMPLEMENTATION PLAN

Work through the phases in order. Phases 1 to 4 build the skeleton, and phases 5 and 6 add the ML and integrations. Adjust time per phase to the days you have. The ML track starts on day one and runs in parallel with the app.

Phase 0: Planning (half day)

- Finalize name, scope and the demo storyline (Sunita and Priya).
- Assign owners: **ML lead and ML support** (six models is too much for one person), backend, mobile, design and data. Suggested ML split: Person A owns models 1 to 3 (chronic risk), Person B owns models 4, 5 and 6 (maternal, PPG quality, cough).
- Create the GitHub repo and a task board. Agree on branch rules (main, dev, feature branches).
- Download all datasets and check their licences in the first days.
- Collect facility sources.

Phase 1: Project setup

Repo structure

```
swasthsetu/
  mobile/        (Expo app)
  backend/       (FastAPI)
  ml/            (notebooks, training scripts, models)
  data/          (seed files: symptoms, facilities, schemes)
  docs/
```

Mobile

```
npx create-expo-app mobile
cd mobile
npx expo install expo-camera expo-av expo-location expo-speech expo-sqlite
expo-secure-store expo-print expo-sharing react-native-webview
npm i @supabase/supabase-js i18next react-i18next zustand @tanstack/react-query
@react-native-voice/voice

eas build:configure
```

Use a development build (`eas build --profile development --platform android`) because voice, camera and audio modules do not all work in Expo Go.

Backend

10. cd backend
11. python -m venv .venv && source .venv/bin/activate
12. pip install fastapi uvicorn pydantic python-jose supabase xgboost scikit-learn shap pandas joblib opencv-python mediapipe scipy numpy python-multipart librosa soundfile pydub

uvicorn app.main:app --reload

Create a `/health` endpoint and deploy a stub early to prove the pipeline works. Install ffmpeg on the server for audio decoding.

Supabase

- Create the project, enable PostGIS, and store keys in `.env` files and Expo secrets.

ML workspace

13. ml/
14. data/raw data/processed
15. notebooks/
16. 01_hypertension.ipynb
17. 02_cvd.ipynb
18. 03_diabetes.ipynb
19. 04_maternal.ipynb
20. 05_ppg_quality.ipynb
21. 06_cough.ipynb
22. src/ (features.py, train.py, evaluate.py, explain.py, export.py)

models/<name>/ (model.joblib, metrics.json, model_card.md)

Standard template for every model: load, clean, split by patient or recording, train baseline and main model, calibrate, evaluate, SHAP or importance, save artifacts, write the model card.

Done when: a “hello” screen calls the deployed `/health` endpoint on a real phone.

Phase 2: Authentication

- Configure Supabase Auth (anonymous sign-in on, email OTP on).
- Build the Language, Consent and Login screens.
- Store the session in secure storage and add an auth guard in navigation.
- Add a backend dependency that verifies the JWT and exposes `current_user`.

Done when: a guest and an email user can both call a protected endpoint.

Phase 3: Database and schemas

- Run the SQL for enums, tables, indexes and RLS from Document 4.
- Seed `symptom_catalog`, `schemes`, `facilities` and `ml_models` (scripts in `data/`).
- Create typed models in the backend (Pydantic) and the app (TypeScript types).
- Test RLS with two users to confirm they cannot see each other's data.

Done when: seed data loads and RLS tests pass.

Phase 4: Core UI and layout

- Build a shared design system: colours, typography, large buttons, icon cards, progress bar, language switcher.
- Set up i18n with the three language files and test switching.
- Build the navigation (stacks and tabs from Document 3).
- Implement all intake screens, including Pregnancy details, with local state and validation.
- Add text-to-speech on each prompt and voice input on the symptoms screen.

Done when: a full intake can be completed in all three languages by voice or tap.

Phase 5: Core features

5a. Red-flag engine (do this first, it is the safety feature)

- Implement rules RF01 to RF10 in TypeScript, including the expanded pregnancy rule, with unit tests for each.
- Build the Emergency Alert screen with call and navigate actions.

5b. ML models (runs in parallel from Phase 1)

Order of work, easiest first, so something always works:

S	Model	Key tasks	Done when
1	CVD risk	Clean Kaggle 70k data, train, calibrate, SHAP	Endpoint returns probability and 3 factors

2	Hypertension	Build “with BP” and “no BP” versions, avoid label leakage	Both variants pass tests
3	Diabetes	Train on BRFSS indicators, compare with the IDRS rule baseline	Metrics and comparison saved
4	Maternal risk	Train on UCI maternal data with cross-validation, build pregnancy rule overrides	High-risk recall reported, rules tested
5	PPG signal quality	Build the feature extractor, label windows from public datasets and your own clips, train the classifier	BPM error with the gate versus without it is measured
6	Cough classifier	Preprocess COUGHVID, extract MFCC features, train XGBoost, optional CNN, build the quality gate	AUC reported and the retry flow works

For each model produce: `model.joblib`, `metrics.json`, a model card (purpose, data, features, metrics, limitations, intended use), and a unit test that checks the output shape.

Data collection tasks for the team (do these early, in parallel)

- Record 40 to 60 face videos with a simultaneous pulse oximeter reading, in different lighting and with different people (for the quality model and rPPG validation). Take consent and delete videos after feature extraction.
- Record 20 to 30 short cough and non-cough samples with the app’s recorder, to test the full pipeline on real phones (for testing, not for training claims).

5c. BP log

- Entry form, local save, sync, trend chart.

5d. rPPG with quality gate

- Prototype extraction and the BPM estimator in a notebook, then add the quality classifier before the BPM step.
- Build the `/vitals/rppg` endpoint returning window labels and reasons.
- Build the camera screen with live guidance, then the “Poor signal” retry screen.
- Run the validation session and produce the “with gate versus without gate” error comparison.

5e. Cough

- Build the audio recorder screen (10 s, level meter, permission handling).
- Build `/vitals/cough` with quality check, cough detection, feature extraction and classification.
- Build the cough result screen and connect the flag to the triage logic.

5f. Maternal module

- Build the Pregnancy details screen and the `maternal_checks` save.
- Connect `/maternal/assess`, the Maternal result card, and the maternity-services facility filter.

5g. Triage logic and result screens

- Implement the combination logic from Document 3, the result cards and risk detail screens.

5h. Facility recommender and schemes

- `/facilities/nearby` with PostGIS, the ranking score and the eligibility rules.
- Map screen with the Leaflet WebView, markers and facility cards.

5i. Reports

- SOAP generator and screen, FHIR Bundle builder and screen (with heart rate quality, cough and maternal fields), and the patient PDF via `expo-print`.
- Validate the FHIR JSON with the HAPI validator.

5j. Model transparency

- `/models` and `/models/{name}/card` endpoints.
- Model Transparency screen listing the six models, their metrics, data sources and limitations.

Done when: the complete Sunita and Priya flows work end to end on a phone.

Phase 6: Integrations and polish

- Connect offline storage and sync (SQLite `synced` flag, retry queue).
- Appointment booking (simulated) and confirmation.
- History screen and BP trends.
- Error, loading and empty states on every screen.
- Add the “simulated” labels, disclaimers and consent text.
- Performance pass: video and audio size limits, API timeouts, caching of facility lists.

Phase 7: Testing

Unit tests

- Red-flag rules (every rule, plus boundary values, including all pregnancy cases).
- Triage logic combinations.
- Output shape and range checks for all six models, SHAP mapping coverage, no missing translations for factor text.
- FHIR builder output.

Model checks

- Leakage checks: confirm train and test splits are separated by patient or recording.
- Maternal: boundary BP values, missing blood sugar and temperature.
- rPPG quality: compare BPM error for Good, Marginal and Poor windows on held-out clips.
- Cough: silence, noise-only, speech-only and real cough clips on at least 3 phones.

Integration tests (pytest and Postman)

- Auth, assessment creation, risk, maternal, rPPG, cough, facilities, SOAP, FHIR, models.

Device testing

- At least 3 real Android phones including a low-end one.
- Airplane mode test for offline features.
- Voice test in Hindi and Marathi with different speakers.
- rPPG test in different lighting and with different people.

Usability test

- 5 or more people (ideally non-technical) complete intake. Note where they get stuck and fix the top issues.

Reporting

- Final metrics tables for all six models, calibration plots, and the rPPG error summary for the slides.

Phase 8: Deployment

- Backend: deploy to Cloud Run, Render or Railway with environment variables, and confirm the HTTPS URL. Make sure ffmpeg and model files are included in the image.
- Database: run migrations on the production Supabase project and re-seed.
- Mobile: build a production APK with EAS (`eas build -p android --profile preview`), install it on test phones, and keep a download link or QR.
- Keep a backup of the APK, the database dump and a screen-recorded demo video.
- Freeze features 24 hours before submission. Only fix bugs after that.

Phase 9: Submission checklist

- Working APK and a live backend.
- GitHub repo with a clean README (setup, architecture, screenshots) and model cards under `ml/models/`.
- 2 to 3 minute demo video following the Sunita story, with a short Priya (maternal) segment.
- Slide deck covering: problem, solution, architecture, a **model summary table** (six models with data source, algorithm and headline metric), rPPG validation, impact, and limitations.
- A **responsible AI slide**: what the models can and cannot do, dataset limitations, why rules back up the maternal model, why the cough result is only a supporting signal.
- Clear labels for simulated data (wait times, booking, parts of the scheme data).
- Only the team leader can submit on the portal, so confirm who submits and test the upload early.

Suggested priority if time runs short

1. Intake plus red flags (including pregnancy rules)
2. CVD, hypertension and diabetes models with SHAP
3. Facility map with scheme info
4. rPPG heart rate with the signal quality classifier
5. Maternal risk model and module
6. Cough classifier
7. PDF, SOAP and FHIR
8. Offline mode and booking mock
9. Model transparency screen

If time gets tight, cut the optional CNN versions first, then the on-device model export. Keep all six models in the demo, but keep the cough model clearly labelled as a supporting screening signal.